

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:30

Annexure A

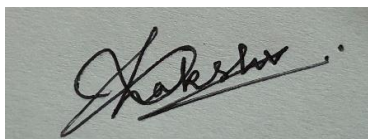
Application Based Questions

Code of Conduct:

I S. LAKSHITA (192524108) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Application Based Questions

1. Intelligent Route Planning using A* and Greedy Search.

A smart delivery system is designed to transport medical supplies between cities. The system uses informed search strategies to determine optimal routes.

The road network is given below:

City	Connected To	Cost	Heuristic $h(n)$ to Goal (G)
A	B, C	2, 4	7
B	D, E	3, 6	6
C	F	5	4
D	G	4	3
E	G	2	2
F	G	3	1
G	—	—	0

Tasks:

- Construct the **search tree** from A to G.
- Apply **Greedy Best-First Search** and show node expansion order.
- Apply **A*** algorithm using $f(n) = g(n) + h(n)$.
- Justify which algorithm is better **for real-time emergency delivery systems**.

2. Constraint Satisfaction Problem – University Timetable Scheduling

A university must schedule final exams for 5 subjects: AI, ML, DBMS, OS, CN across 3 days (Day 1, Day 2, Day 3) with the conditions, AI and ML cannot be scheduled on the same day, DBMS must be scheduled before OS, CN cannot be on Day 1, At most 2 exams per day, A student is enrolled in AI, DBMS, and CN → these cannot overlap

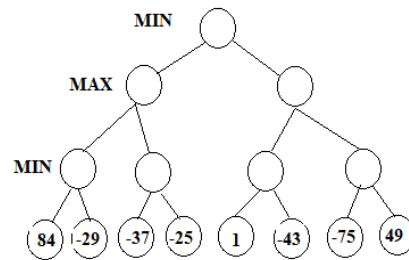
Tasks:

- Formulate this problem as a Constraint Satisfaction Problem (CSP) by defining, Variables, Domains and Constraints
- Draw the constraint graph.
- Solve using Backtracking Search.
- Apply Forward Checking or Arc Consistency and explain how it reduces the search space.

- v. Provide one valid timetable solution.

3. Game Playing – Minimax and Alpha-Beta Pruning

An AI agent is designed to play a turn-based strategy game. The game tree is given below:



Tasks:

- Apply the **Minimax algorithm** and compute values for all internal nodes.
- Identify the **optimal move for MAX**.
- Apply **Alpha-Beta Pruning** to Show α and β values at each step and Clearly indicate which nodes are pruned
- Compare nodes evaluated in Minimax vs Alpha-Beta
- Explain the importance of **evaluation functions** in complex games like chess.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

CO2 – ASSESSMENT TOOL 1
Application Based Questions

COURSE NAME: Artificial Intelligence

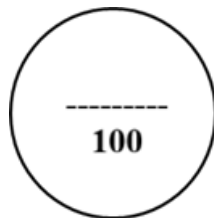
COURSE CODE: CSA17

QUESTIONS: 3

Duration: 1 Hour

Total Marks: 30

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Total Marks (30)
Understanding of Problem Context	20% (2)	/2	/2	/2	/6
Application of Concepts	30% (3)	/3	/3	/3	/9
Problem-Solving Approach	20% (2)	/2	/2	/2	/6
Accuracy of Solution	20% (2)	/2	/2	/2	/6
Clarity	10% (1)	/1	/1	/1	/3
Total per Question	10 Marks	/10	/10	/10	/30



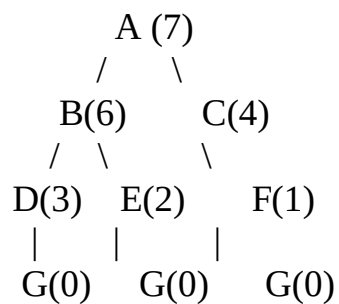
ANSWERS

1. Intelligent Route Planning using A* and Greedy Search.

Given Data

City	Connected To	Cost	Heuristic h(n)
A	B, C	2, 4	7
B	D, E	3, 6	6
C	F	5	4
D	G	4	3
E	G	2	2
F	G	3	1
G	—	—	0

i. Search Tree from A to G



ii. Greedy Best-First Search

Evaluation Function

$$f(n) = h(n)$$

The node having the smallest heuristic value is expanded first.

Step	Open List	Selected Node
1	A	A
2	B(6), C(4)	C
3	B(6), F(1)	F
4	B(6), G(0)	G (Goal)

Node Expansion Order

$$A \rightarrow C \rightarrow F \rightarrow G$$

Path Found

$$A \rightarrow C \rightarrow F \rightarrow G$$

Total Cost

$$4 + 5 + 3 = 12$$

iii. Apply A* Algorithm**Evaluation Function**

$$f(n) = g(n) + h(n)$$

where

- $g(n)$ = actual path cost from start
- $h(n)$ = heuristic estimate to goal

Step 1

Start at A

Node	g	h	f
A	0	7	7

Expand A.

Step 2

Generate successors.

Node	g	h	f
B	2	6	8
C	4	4	8

Choose **B** (tie broken by first generated).

Step 3

Expand B.

Node	g	h	f
C	4	4	8
D	5	3	8
E	8	2	10

Choose C.

Step 4

Expand C.

Node	g	h	f
D	5	3	8

E	8	2	10
F	9	1	10

Choose **D**.

Step 5

Expand D.

Node	g	h	f
E	8	2	10
F	9	1	10
G	9	0	9

Choose **G**.

Goal reached.

Node Expansion Order

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$

Optimal Path

$A \rightarrow B \rightarrow D \rightarrow G$

Total Cost

$$2 + 3 + 4 = 9$$

iv. Justification

Greedy Best-First Search	A* Search
Uses only heuristic $h(n)$	Uses both $g(n)$ and $h(n)$
Usually faster	Slightly slower
May produce a non-optimal path	Produces the optimal path (with an admissible heuristic)
Path found: $A \rightarrow C \rightarrow F \rightarrow G$ (Cost = 12)	Path found: $A \rightarrow B \rightarrow D \rightarrow G$ (Cost = 9)

For real-time emergency delivery systems, A* is the better algorithm because it considers both the distance already traveled and the estimated distance to the goal. This enables it to find the shortest and most cost-effective route, making it more

reliable than Greedy Best-First Search, which may choose a quicker-looking but more expensive path.

2. Constraint Satisfaction Problem (CSP) – University Timetable Scheduling

i. Formulate the Problem as a CSP

Variables

The variables represent the subjects to be scheduled.

$$[X = \{AI, \backslash ML, \backslash DBMS, \backslash OS, \backslash CN\}]$$

Domains

Each subject can be scheduled on any one of the three exam days.

Variable	Domain
AI	{Day 1, Day 2, Day 3}
ML	{Day 1, Day 2, Day 3}
DBMS	{Day 1, Day 2, Day 3}
OS	{Day 1, Day 2, Day 3}
CN	{Day 1, Day 2, Day 3}

Constraints

1. AI and ML cannot be scheduled on the same day.

$$AI \neq ML$$

2. DBMS must be scheduled before OS.

$$DBMS < OS$$

3. CN cannot be on Day 1.

$$CN \neq Day\ 1$$

4. At most two exams can be conducted on each day.

Maximum 2 subjects per day

5. A student is enrolled in AI, DBMS and CN; therefore these exams cannot overlap.

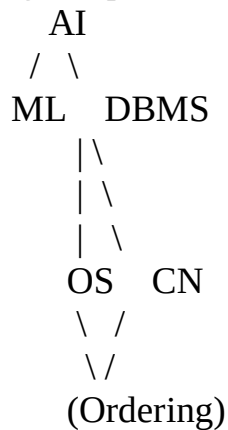
$$AI \neq DBMS$$

$$AI \neq CN$$

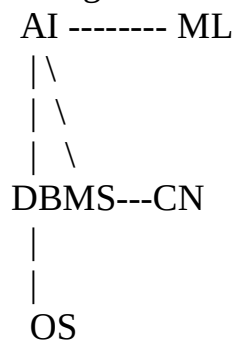
$$DBMS \neq CN$$

ii. Constraint Graph

The edges represent constraints between variables.



Or, showing all constraints clearly:



Edges:

- AI — ML
- AI — DBMS
- AI — CN
- DBMS — CN
- DBMS — OS (ordering constraint)

iii. Solution using Backtracking Search

Step 1

Assign

$$AI = \text{Day 1}$$

Step 2

ML cannot be Day 1.

Choose

$$ML = \text{Day 2}$$

Step 3

DBMS cannot be Day 1 (AI already there).

Choose

$$DBMS = Day\ 2$$

Step 4

OS must be after DBMS.

Choose

$$OS = Day\ 3$$

Step 5

CN cannot be Day 1.

Cannot be Day 2 (same student as DBMS).

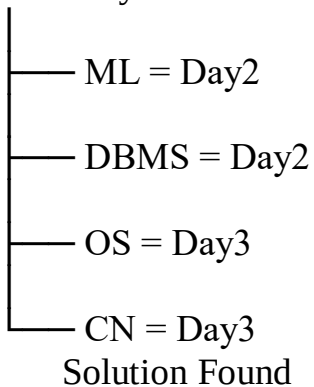
Hence,

$$CN = Day\ 3$$

All constraints are satisfied.

Backtracking Search Tree

AI = Day1



No backtracking is required because each assignment satisfies all constraints.

iv. Forward Checking

Forward Checking removes inconsistent values from the domains immediately after assigning a variable.

Example

Initially,

Subject	Domain
AI	{D1,D2,D3}
ML	{D1,D2,D3}
DBMS	{D1,D2,D3}
OS	{D1,D2,D3}
CN	{D1,D2,D3}

Assign

$$AI = Day\ 1$$

Updated domains:

- $ML \rightarrow \{Day2, Day3\}$
- $DBMS \rightarrow \{Day2, Day3\}$
- $CN \rightarrow \{Day2, Day3\}$

Assign

$$DBMS = Day\ 2$$

Updated domains:

- $OS \rightarrow \{Day3\}$
- $CN \rightarrow \{Day3\}$

Since impossible values are removed early, the algorithm avoids exploring invalid branches and reaches the solution faster.

Search Space Reduction:

- Eliminates inconsistent assignments immediately.
- Prevents unnecessary backtracking.
- Improves efficiency by reducing the number of possible combinations explored.

v. One Valid Timetable

Day	Subjects
Day 1	AI
Day 2	ML, DBMS
Day 3	OS, CN

Final Valid Timetable

Day	Exams
Day 1	AI
Day 2	ML, DBMS
Day 3	OS, CN

3. Game Playing – Minimax and Alpha-Beta Pruning

The game tree has three levels:

- **Root:** MIN
- **Level 1:** MAX
- **Level 2:** MIN
- **Leaf Nodes:** 84, -29, 37, -25, 1, -43, -75, 49

i. Apply the Minimax Algorithm

Step 1: Compute MIN Nodes (Bottom Level)

Node A

$$\min(84, -29) = -29$$

Node B

$$\min(37, -25) = -25$$

Node C

$$\min(1, -43) = -43$$

Node D

$$\min(-75, 49) = -75$$

Step 2: Compute MAX Nodes

Left MAX

$$\max(-29, -25) = -25$$

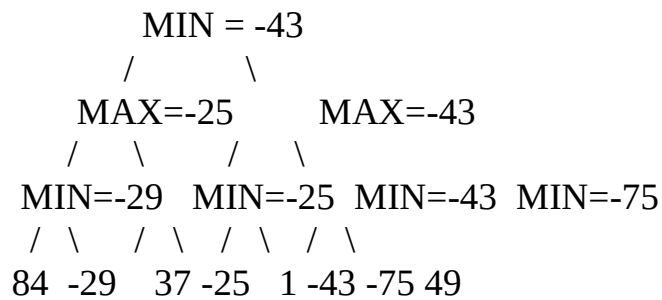
Right MAX

$$\max(-43, -75) = -43$$

Step 3: Compute Root (MIN)

$$\min(-, 25 - 43) = -43$$

Final Minimax Tree



ii. Optimal Move for MAX

The **MAX player** would select:

- **Left branch** → value = -25
- **Right branch** → value = -43

At the root (MIN), the opponent chooses the **right branch**, giving the final outcome **-43**.

iii. Alpha-Beta Pruning

Initially,

$$\alpha = -\infty, \beta = +\infty$$

Root (MIN)

$$\alpha = -\infty$$

$$\beta = +\infty$$

Explore Left MAX

$$\alpha = -\infty$$

$$\beta = +\infty$$

Left MIN

Leaves:

84, -29

Value = -29

Return -29

MAX updates

$$\alpha = \max(-\infty, -29) = -29$$

Second MIN

Leaves:

37, -25
Value= -25

Return -25
MAX updates
 $\alpha = \max(-29, -25) = -25$
Left MAX returns -25
Root updates
 $\beta = \min(+\infty, -25) = -25$

Explore Right MAX

Current values
 $\alpha = -\infty$
 $\beta = -25$

Third MIN

Leaves:
1, -43
Value= -43

Return -43
MAX updates
 $\alpha = \max(-\infty, -43) = -43$
Since

$$\alpha(-43) < \beta(-25)$$

Continue searching.

Fourth MIN

Current
 $\alpha = -43$
 $\beta = -25$
First leaf
-75
Current MIN value
 $\beta = -75$
Now

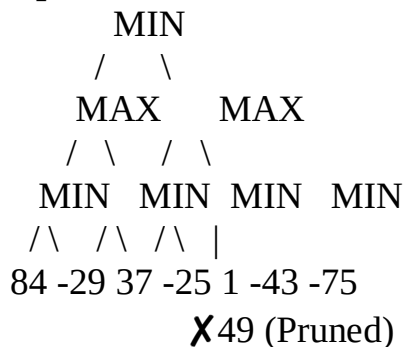
$$\beta = -75 \leq \alpha = -43$$

Therefore,
Prune remaining child (49).

Pruned Node

49

Final Alpha-Beta Tree



α and β Values

Node	α	β
Root (MIN)	$-\infty$	-25
Left MAX	-25	$+\infty$
Right MAX	-43	-25
Last MIN	-43	-75 (Cutoff)

iii. Comparison of Nodes Evaluated

Algorithm	Leaf Nodes Evaluated	Nodes Pruned
Minimax	8	0
Alpha-Beta	7	1 (49)

Comparison:

- Minimax evaluates all 8 leaf nodes.
- Alpha-Beta Pruning evaluates only 7 leaf nodes by pruning the leaf 49.
- Alpha-Beta produces the same optimal result as Minimax while reducing the number of nodes explored, making it more efficient.

v. Importance of Evaluation Functions in Complex Games

An evaluation function estimates the quality of a game position when it is impractical to search the entire game tree (as in chess).

Importance

- Estimates the strength of non-terminal game states.
- Enables AI to make good decisions without exploring every possible move.
- Reduces computation time by limiting deep searches.
- Works with Minimax and Alpha-Beta to identify promising moves efficiently.

- Essential in complex games like chess, where the number of possible positions is extremely large.

Examples in Chess

An evaluation function may consider:

- Material advantage (piece values)
- King safety
- Control of the center
- Pawn structure
- Piece mobility
- Threats and tactical opportunities

These factors help the AI choose strong moves even when the game cannot be searched to completion.